DOKUMEN TEKNIS & ARSITEKTUR SISTEM

# LAPORAN ARSITEKTUR & PANDUAN PENGEMBANGAN SISTEM MONITORING PLTS

Studi Kasus Integrasi Multi-Vendor (Huawei FusionSolar & Kehua Tech), Otomasi Web Scraping API, Pemrosesan Data Telemetri, dan Dashboard Terpadu

**Kategori:** Arsitektur Perangkat Lunak & IoT Energi Terbarukan • **Format:** Laporan Teknis Resmi • **Tanggal:** 2026

---

## BAB 1. PENDAHULUAN & LATAR BELAKANG

Pembangkit Listrik Tenaga Surya (PLTS) pada skala komersial, gedung kampus, atau industri umumnya menggunakan perangkat inverter dari berbagai vendor terpisah. Pada studi kasus proyek ini (seperti pada kampus Politeknik Negeri Jakarta), beberapa gedung menggunakan inverter buatan **Huawei**, sedangkan gedung lainnya menggunakan inverter buatan **Kehua Tech**.

Tantangan utama yang dihadapi oleh operator teknis adalah *data fragmentation* (fragmentasi data). Masing-masing vendor memiliki portal cloud tersendiri:

- **Huawei FusionSolar:** `https://sg5.fusionsolar.huawei.com`
- **Kehua Energy Cloud:** `https://energy.kehua.com`

Kondisi ini menyulitkan pengawasan terpusat: operator tidak bisa melihat total daya instan gabungan (*fleet capacity*), total pendapatan penghematan listrik harian, perbandingan efisiensi antar gedung, serta status gangguan secara langsung tanpa membuka dan mencatat dari dua situs web yang berbeda secara manual.

**Tujuan Pembangunan Sistem:**

Membuat satu sistem monitoring terpusat (*Single Pane of Glass*) yang bekerja secara otonom tanpa campur tangan manusia untuk:

1. Menarik data operasional (daya kW, energi harian kWh, status alarm) dari kedua vendor secara berkala.
2. Menstandarisasi format data yang heterogen ke dalam basis data tunggal (PostgreSQL/Supabase).
3. Menyajikan dashboard web real-time yang dilengkapi peta interaktif lokasi gedung dan analisis finansial.

## BAB 2. ARSITEKTUR SISTEM MENYELURUH (END-TO-END)

Sistem monitoring ini dirancang menggunakan arsitektur 4-lapisan (*4-Tier Decoupled Architecture*). Pemisahan ini menjamin apabila salah satu portal vendor mengalami gangguan, dashboard tetap aktif dan menampilkan data historis terakhir.


1. FIELD LEVEL (PERANGKAT LAPANGAN)
Inverter Huawei & Kehua di Atap Gedung --> Mengirim data via WiFi/4G Dongle
2. VENDOR CLOUD PORTAL
Huawei FusionSolar Cloud (SG5) & Kehua Energy Cloud Platform
[Otomasi Scraper / API Replay via Python Requests]
3. BACKEND & INTEGRATION ENGINE (FastAPI - Python)
- Auth Manager: Enkripsi AES-CBC Kehua, Cookie Handshake Huawei
- Data Scraper: Mengunduh JSON real-time dari internal endpoint vendor
- Normalizer : Mengonversi data berbeda format ke schema 'monitoring_current.v2'
- Webhook Refresher: Endpoint POST /api/refresh (Dipicu Cron Scheduler)
[Menyimpan status live & time-series history]
4. DATABASE LAYER (Supabase / PostgreSQL)
- monitoring_current : 1 baris status telemetri terbaru seluruh stasiun
- monitoring_snapshots: Rekaman riwayat snapshot tiap jam (retensi 30 hari)
- monitoring_site_daily: Rekaman agregasi harian untuk grafik jangka panjang
[HTTP GET /api/current & /api/history via JSON]
5. FRONTEND DASHBOARD (Vite + React.js)
- Overview Tab: Total Fleet KPI (kWp, kW saat ini, kWh bulanan, Estimasi IDR)
- Energy Tab : Grafik kurva daya 5-menit, harian bulan ini, bulanan tahun ini
- Revenue Tab : Grafik nominal pendapatan & penghematan tagihan listrik PLN
- GIS Map : Peta satelit interaktif sebaran lokasi stasiun (MapLibre GL)

# BAB 3. MEKANISME PENGAMBILAN DATA OTOMATIS (THE SCRAPER ENGINE)

Bagian ini menjawab secara rinci bagaimana data dari portal vendor diambil secara otomatis meskipun vendor tidak menyediakan Open API publik yang mudah diakses.

## 3.1. Teknik Reverse-Engineering Internal API (Blueprint Templates)

Sistem tidak menggunakan browser berbasis antarmuka visual berat seperti Selenium/Puppeteer (yang boros memori RAM), melainkan menggunakan teknik **HTTP API Replay** menggunakan library Python `requests` murni.

Pada berkas `backend/huawei_api_blueprint.json` dan `backend/kehua_api_blueprint.json`, tersimpan cetak biru (*blueprint*) dari rekaman Network HTTP saat browser memuat dashboard vendor resmi. Berkas ini mencatat URL asli, query parameters, dan struktur header yang dibutuhkan.

## 3.2. Penanganan Autentikasi Mandiri (Token Renewal & Security Bypass)

Token atau sesi login pada portal web memiliki masa kedaluwarsa (biasanya beberapa jam). Agar script dapat berjalan tanpa henti (24/7), script `backend/scrape_monitoring.py` dilengkapi mekanisme login otonom:

| Vendor | Tantangan Keamanan | Solusi Otomasi Backend Python |
|---|---|---|
| **Kehua Tech** ( `energy.kehua.com` ) | Password pengguna tidak dikirim sebagai plain text. Portal Kehua menggunakan enkripsi AES pada frontend JavaScript dan validasi signature header. | Backend mereplikasi algoritma enkripsi Kehua secara lokal:<br>• Enkripsi kata sandi menggunakan **AES-CBC** dengan key: `clxslychxyl11988`.<br>• Membuat tanda tangan digital (hash signature) pada header menggunakan secret: `%vSBtRMV4F8mr#dYofsG3lJOBv5vw*fZ`.<br>• Mengirim ke endpoint `/necp/server-user/auth/web/login` dan menyimpan `token` baru di memory RAM. |
| **Huawei FusionSolar** ( `fusionsolar.huawei.com` ) | Sistem menggunakan sesi berbasis cookie terenkripsi ( `RoaRand` dan `SESSION` ) serta validasi domain aktif. | • Sebelum scraping, backend memanggil endpoint pengecekan: `/rest/dpcloud/auth/v1/is-session-alive`.<br>• Jika respon bernilai false, sistem melakukan login ulang menggunakan kredensial yang tersimpan di environment variables ( `HUAWEI_USERNAME` & `HUAWEI_PASSWORD` ).<br>• Melakukan periodic keep-alive untuk memperpanjang usia sesi. |

## 3.3. Endpoint Telemetri yang Diambil

Setelah sesi terverifikasi, backend Python mengeksekusi serangkaian panggilan API internal:

- **Pada Huawei:**
  - `/rest/pvms/web/station/v1/station/station-list`: Mengambil metadata seluruh stasiun (nama, kapasitas terpasang kWp, koordinat garis lintang dan bujur).

- `/rest/pvms/web/station/v1/station/total-real-kpi` : Mengambil daya aktual saat ini (*real-time active power*) dan energi hari ini.
- `/rest/pvms/web/report/v1/station/home-station-kpi-chart` : Mengambil kurva produksi energi resolusi tinggi (tiap 5 menit) untuk grafik kurva lonceng radiasi matahari.

- **Pada Kehua:**
  - `/server-maintenance/station` : Mengambil daftar inventaris stasiun dan status konektivitas.
  - `/server-monitor/monitor` : Mengambil pembacaan sensor inverter (tegangan AC/DC, arus, temperatur, frekuensi grid).
  - `/server-maintenance/report/deviceData` : Mengambil data riwayat energi harian dan bulanan.

### 3.4. Normalisasi Data (Data Normalization)

Karena masing-masing vendor menggunakan terminologi dan struktur JSON yang sangat berlainan, backend mentransformasikan seluruh respon ke dalam skema standar seragam:

```
{
  "schema": "monitoring_current.v2",
  "updated_at": "2026-09-30T09:40:00Z",
  "by_site": {
    "huawei": {
      "platform": "huawei",
      "overview": { "capacity_kwp": 120.0, "current_power_kw": 85.4, "daily_energy_kwh": 412.5 },
      "stations": [ ... ]
    },
    "kehua": {
      "platform": "kehua",
      "overview": { "capacity_kwp": 95.0, "current_power_kw": 42.1, "daily_energy_kwh": 198.4 },
      "stations": [ ... ]
    }
  }
}
```

# BAB 4. PENJADWALAN & PENYIMPANAN DATA (DATABASE PIPELINE)

## 4.1. Mekanisme Pemicu (Scheduler / Cron)

Proses scraping tidak dijalankan secara manual oleh pengguna, melainkan digerakkan oleh penjadwal otomatis:

- Layanan penjadwal cloud (seperti `cron-job.org` atau Render Background Worker) mengirim HTTP request berkala:

```
POST https://api-backend-anda.com/api/refresh
Header: Authorization: Bearer <REFRESH_SECRET>
```

- Endpoint ini dilindungi oleh token rahasia ( `REFRESH_SECRET` ) agar tidak disalahgunakan pihak luar.
- Begitu request diterima, backend Python menjalankan fungsi penarikan data ke vendor, memproses normalisasi, dan memperbarui database.

## 4.2. Struktur Basis Data (Supabase PostgreSQL)

Supabase digunakan sebagai penyimpanan data cloud yang tangguh. Skema database terdiri dari:

| Nama Tabel | Frekuensi Tulis | Fungsi Utama |
|---|---|---|
| `monitoring_current` | Setiap 5-10 menit (Upsert 1 baris) | Menyimpan kondisi terkini seluruh stasiun untuk ditampilkan langsung saat pengguna membuka halaman pertama kali. |
| `monitoring_snapshots` | Setiap 1 jam (Append log) | Menyimpan rekam jejak historis untuk melihat tren performa. Data lama dihapus otomatis sesuai kebijakan retensi (misal 30 hari). |
| `monitoring_site_daily` | Setiap akhir hari (Rollup) | Menyimpan akumulasi produksi kWh dan pendapatan per hari per stasiun untuk pelaporan jangka panjang bertahun-tahun. |

# BAB 5. DESAIN ANTARMUKA PENGGUNA (FRONTEND DASHBOARD)

Frontend dibangun dengan framework **React (Vite)** yang mengutamakan performa tinggi dan estetika modern:

- **Sistem Desain Dark Mode & Glassmorphism:** Memberikan tampilan profesional bergaya pusat kontrol (*command center*) dengan kartu berbayang, aksen neon indikator status online/offline, dan kontras visual tinggi.
- **Visualisasi Peta Geospasial (GIS):** Menggunakan library **MapLibre GL** yang terhubung ke ubin peta OpenFreeMap. Setiap stasiun PLTS ditandai dengan pin interaktif berdasarkan titik koordinat GPS aktual (lintang & bujur), menampilkan pop-up kapasitas saat diklik.
- **Grafik Performa Real-Time:**
  - *Intraday Solar Curve:* Grafik SVG responsif yang menggambarkan kenaikan radiasi matahari dari jam 06:00 pagi hingga penurunan di sore hari.
  - *Financial Analytics:* Konversi kWh ke estimasi rupiah (menggunakan formula tarif dasar listrik industri/sosial PLN per kWh).

## BAB 6. PANDUAN PRAKTIS: CARA MEMBANGUN SISTEM SEPERTI INI DARI NOL

Jika Anda berencana mengembangkan website monitoring PLTS serupa secara mandiri, berikut adalah peta jalan (*roadmap*) teknis langkah demi langkah yang dapat Anda ikuti:

### LANGKAH 1: Tentukan Arsitektur Akuisisi Data Lapangan

Terdapat 2 opsi untuk mendapatkan data dari inverter:

- **Opsi A (Melalui Cloud Vendor - Seperti Proyek Ini):**
  Cocok bila inverter sudah terhubung ke internet dan Anda memiliki akun portal web vendor. Gunakan Chrome DevTools (Network tab) untuk melihat bagaimana web tersebut memuat data, lalu buat script Python untuk meniru request-nya.
- **Opsi B (Langsung dari Hardware via IoT Hardware Gateway):**
  Bila Anda ingin data lokal tanpa bergantung pada portal vendor, pasang mikrokontroler (ESP32 atau Raspberry Pi) yang dihubungkan ke port **RS485 Modbus RTU** inverter. Script membaca register tegangan/daya inverter, lalu mengirim data ke server Anda via protokol **MQTT** atau **HTTP POST**.

### LANGKAH 2: Bangun REST API Backend (Python FastAPI)

1. Gunakan **FastAPI** karena sangat cepat, ringan, dan otomatis menyediakan dokumentasi interaktif Swagger UI ( `/docs` ).
2. Gunakan library `requests` atau `httpx` untuk melakukan penarikan data dari vendor.
3. Gunakan `pydantic` untuk memvalidasi dan memformat data JSON agar selalu konsisten sebelum disimpan ke database.

### LANGKAH 3: Siapkan Database Cloud (Supabase / PostgreSQL)

1. Buat proyek gratis di supabase.com.
2. Buat tabel untuk data instan ( `current` ) dan data riwayat ( `history` ).
3. Gunakan library resmi `supabase-py` pada backend untuk proses `select()` , `insert()` , dan `upsert()` .

### LANGKAH 4: Kembangkan Dashboard Web (React / Vue)

1. Inisialisasi project menggunakan Vite: `npm create vite@latest dashboard-plts -- --template react` .
2. Gunakan pustaka grafik modern: **Recharts**, **Chart.js**, atau **Apache ECharts** untuk kurva energi.
3. Gunakan pustaka peta: **Leaflet** (via `react-leaflet` ) atau **MapLibre GL**.
4. Atur auto-polling (misalnya menggunakan `setInterval` atau React Query) setiap 60 detik agar data di layar selalu sinkron dengan kondisi di lapangan.

**LANGKAH 5: Publikasi & Otomasi (Deployment)**

- **Backend:** Deploy ke layanan seperti Render, Railway, atau VPS Ubuntu.
- **Frontend:** Deploy ke Vercel, Netlify, atau Cloudflare Pages (gratis dan cepat).
- **Penjadwal (Cron):** Daftarkan URL backend `/api/refresh` ke layanan seperti `cron-job.org` agar scraper otomatis berjalan setiap 5 menit.

## BAB 7. KESIMPULAN

Proyek sistem monitoring PLTS terpadu ini mendemonstrasikan bagaimana integrasi perangkat lunak modern dapat menjembatani keterbatasan sistem tertutup milik vendor perangkat keras (*hardware vendor lock-in*). Dengan memanfaatkan teknik *reverse-engineering API*, otomatisasi enkripsi sesi, standardisasi skema data, serta antarmuka web interaktif, sistem ini berhasil menghadirkan visibilitas energi surya yang lengkap, efisien, dan transparan.

---